



Final Project

Database management system for Public Library

Name: Narra Venkata Naga Sanjay Reddy

G-Number: G01425316

Course: CS504

Section: 003

Course Advisor: Binqian Yin

12 April,2024

2. Database Design

2.1 Define the scope of the project and identify the entities and their relationships.

The project provides an all-inclusive solution designed to satisfy the changing requirements of modern libraries. It offers a strong platform made to successfully manage resources and interact with users. This project intends to improve user experience and streamline library operations by integrating multiple modules, including staff management, authorship attribution, borrowing systems, member administration, and material management. Books, periodicals, e-books, and audiobooks are just a few of the things that librarians can effortlessly arrange and manage. They may also effortlessly monitor loan durations and borrowing activity. The technology also makes it easier to handle member accounts and author information, encouraging acknowledgment of creative efforts and providing library users with individualized services.

The membership status control mechanism of the Library Management project is an interesting aspect. It automatically deactivates memberships if fees are past due and reactivates them after the arrears are paid. This function keeps users accountable by guaranteeing equitable use of the library's resources and enhancing membership management. This project, in its whole, represents a comprehensive approach to contemporary library management, with the goals of optimizing resource use, improving customer happiness, and equipping library staff with effective tools for more efficient operations. The project is well-positioned to transform library services in the digital age, bringing in a new era of efficiency and efficacy in library management with its extensive features and functionalities.

Entities and Relationships:

i. **Material**

Represents individual items available in the library, such as books, magazines, e-books, and audiobooks.

Attributes:

- **Material_ID:** A unique identifier for each material.
- **Title:** The title of the material.
- **Publication_Date:** The date of publication of the material.
- **Catalog_ID:** A reference to the catalog entry for the material.
- **Genre_ID:** A reference to the genre of the material.

ii. **Catalog**

Represents a record of library materials with information on their availability and location.

Attributes:

- **Catalog_ID:** A unique identifier for each catalog entry.
- **Name:** The name of the catalog.
- **Location:** The location of the material within the library.

iii. **Genre**

Represents the various genres or categories of library materials.

Attributes:

- **Genre_ID:** A unique identifier for each genre.
- **Name:** The name of the genre.
- **Description:** The brief introduction of the genre.

iv. **Borrow**

Represents the borrowing activity of library materials by members.

Attributes:

- Borrow_ID: A unique identifier for each borrowing transaction.
- Material_ID: A reference to the borrowed material.
- Member_ID: A reference to the member who borrowed the material.
- Staff_ID: A reference to the staff who processed the transaction.
- Borrow_Date: The date the material was borrowed.
- Due_Date: The date the material is due.
- Return_Date: The date the material is returned.

v. Author

Represents authors who have created library materials.

Attributes:

- Author_ID: A unique identifier for each author.
- Name: The name of the author.
- Birth_Date: The birth date of the author.
- Nationality: The nationality of the author.

vi. Authorship

Represents the relationship between authors and the materials they have created.

Attributes:

- Authorship_ID: A unique identifier for each authorship record.
- Author_ID: A reference to the author.
- Material_ID: A reference to the material authored.

vii. Member

Represents library members who can borrow and reserve materials.

Attributes:

- Member_ID: A unique identifier for each member.
- Name: The name of the member.
- Contact_Info: Email address (or phone number) of the member.
- Join_Date: The date the member joined the library.

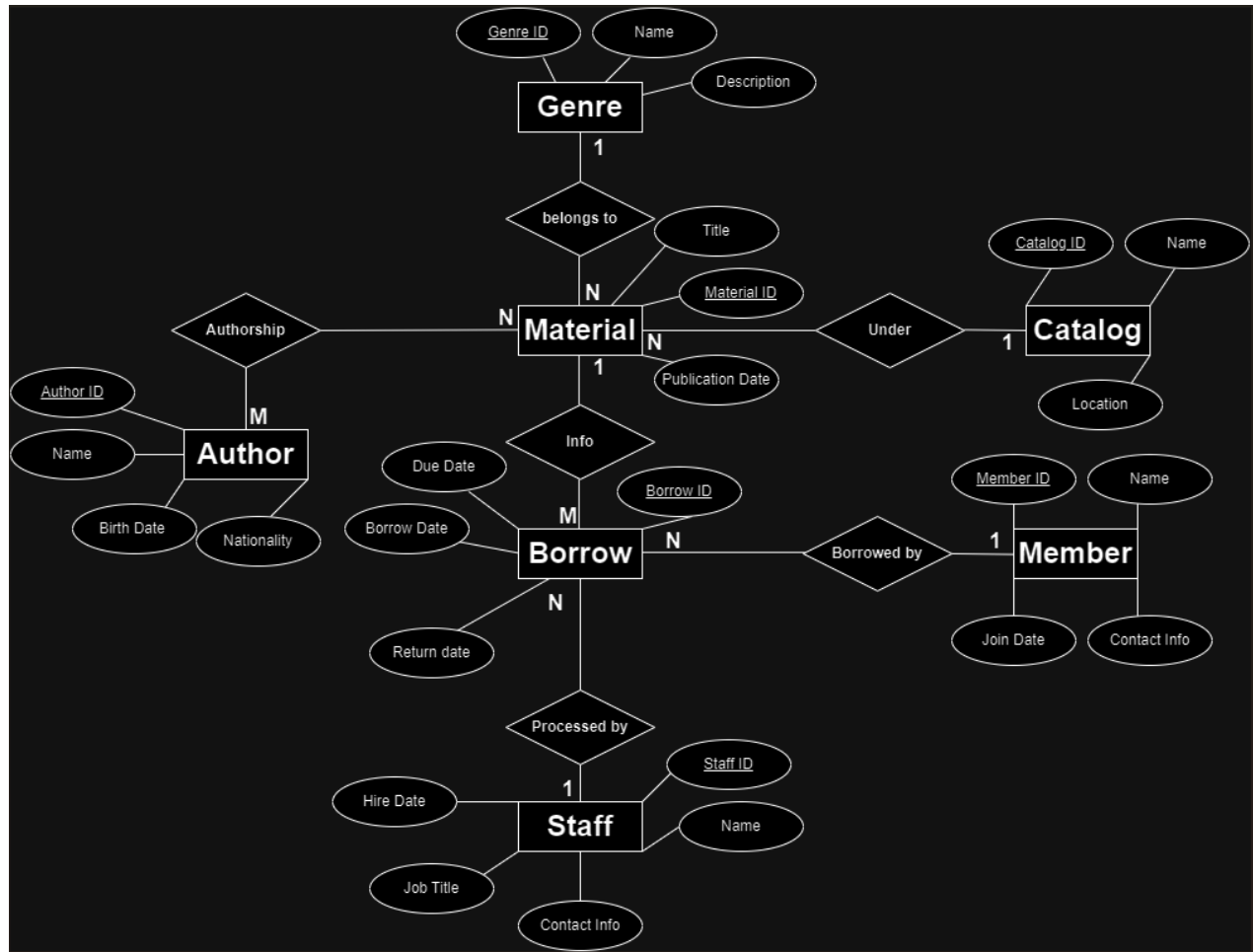
viii. Staff

Represents library staff who manage library resources and assist members.

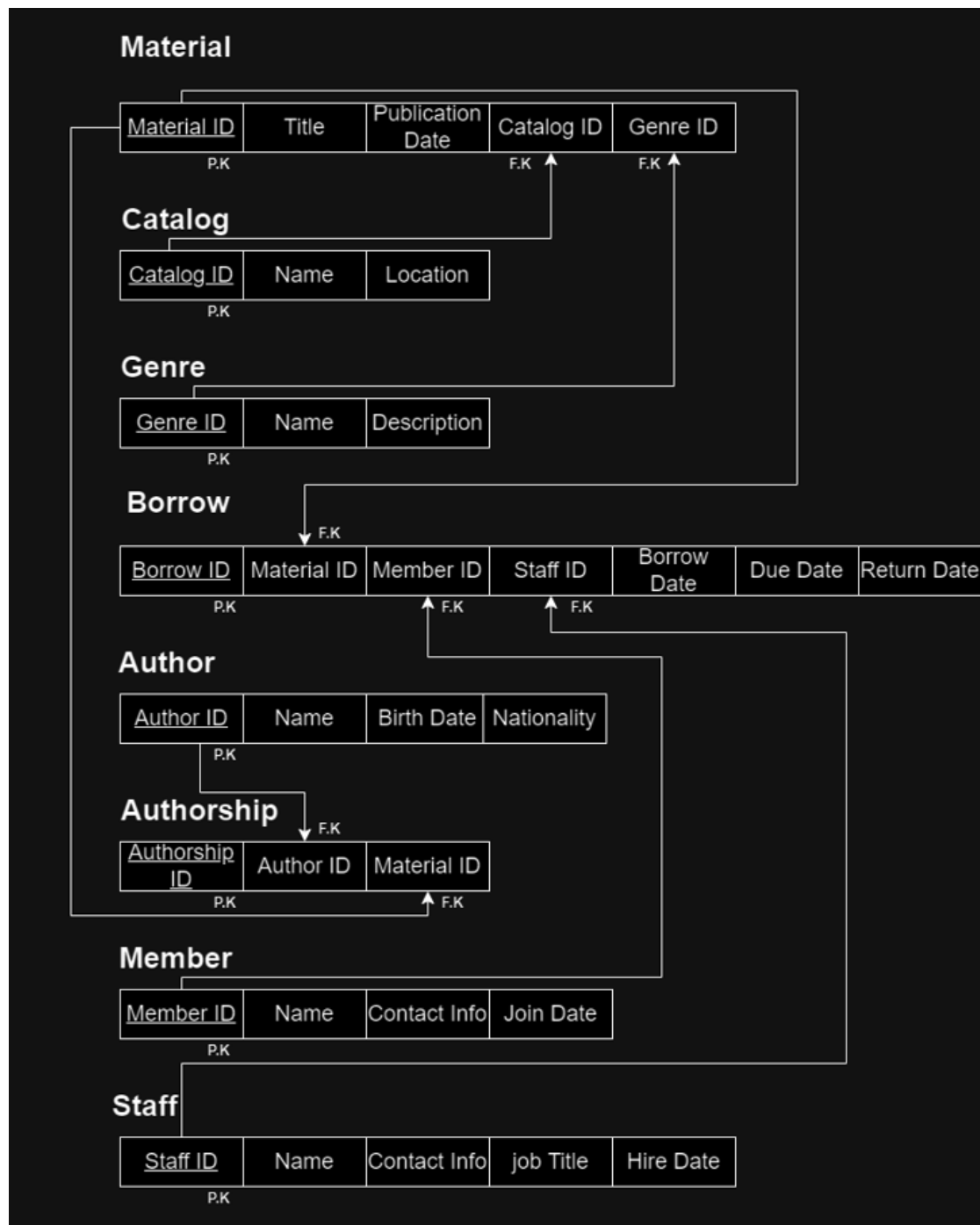
Attributes:

- Staff_ID: A unique identifier for each staff member.
- Name: The name of the staff member.
- Contact_Info: Email address (or phone number) of the member.
- Job_Title: The jobtitle of the staff member (e.g., librarian, assistant librarian).
- Hire_Date: The date the staff member was hired by the library.

2.2 Create an Entity-Relationship (ER) diagram to represent the database schema.



2.3 Mapping the ER diagram to relational schema.



Entities:

- 1) Material
- 2) Borrow
- 3) Staff
- 4) Catalog
- 5) Member
- 6) Genre
- 7) Author

Relationships and Cardinality Ratios:

Relation	Cardinality Ratio	Entities Involved
belongs to	1: N	Genre, Material
Info	1: M	Material, Borrow
Processed by	N: 1	Borrow, Staff
Under	N: 1	Material, Catalog
Borrowed by	N: 1	Borrow, Member
Authorship	N: M	Material, Author

3. Database Implementation

3.1 Choose and install an appropriate Database Management System (DBMS) for this project. We recommend using PostgreSQL.

I decided to use Visual Studio as the IDE and MySQL as the Database Management System (DBMS) for this project. I went with MySQL instead of PostgreSQL even though it was suggested, since it fits my experience and project requirements better. Visual Studio delivers an easy-to-use development environment, and MySQL offers robust database management capabilities.

3.2 Implement the database schema using SQL or the chosen DBMS's data definition language (DDL)

➤ **Creating databases and tables as per data of the library management system.**

❖ **Creating a Database and using it**

```
Create DATABASE Sanjayproj;
```

```
USE Sanjayproj;
```

❖ **Creating Catalog Table**

```
CREATE TABLE catalog(  
    Catalog_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Name VARCHAR(50) NOT NULL,  
    Location VARCHAR(50) NOT NULL  
);
```

❖ **Creating Genre Table**

```
CREATE TABLE Genre (  
Genre_ID INT PRIMARY KEY AUTO_INCREMENT ,  
Name VARCHAR(255),  
Description TEXT  
);
```

❖ **Creating Author table**

```
CREATE TABLE Author (  
Author_ID INT PRIMARY KEY AUTO_INCREMENT ,  
Name VARCHAR(255),  
Birth_Date DATE ,  
Nationality VARCHAR(255)  
);
```

❖ **Creating Member table**

```
CREATE TABLE Member (  
Member_ID INT PRIMARY KEY AUTO_INCREMENT,  
Name VARCHAR(255) ,  
Contact_Info VARCHAR(255) ,  
Join_Date DATE  
);
```

❖ **Creating Staff table**

```
CREATE TABLE Staff (  
Staff_ID INT PRIMARY KEY AUTO_INCREMENT,  
Name VARCHAR(255) ,  
Contact_Info VARCHAR(255) ,  
Job_Title VARCHAR(255) ,  
Hire_Date DATE  
);
```

❖ **Creating Material Table**

```
CREATE TABLE Material (  
Material_ID INT PRIMARY KEY,  
Title VARCHAR(255),  
Publication_Date DATE ,  
Catalog_ID INT ,  
Genre_ID INT,  
CONSTRAINT fcatalog FOREIGN KEY (Catalog_ID) REFERENCES Catalog(Catalog_ID),  
CONSTRAINT fGenre FOREIGN KEY (Genre_ID) REFERENCES Genre(Genre_ID)  
);
```

❖ **Creating Authorship Table**

```
CREATE TABLE Authorship (  
Authorship_ID INT PRIMARY KEY,  
Author_ID INT ,  
Material_ID INT ,  
CONSTRAINT fAuthor FOREIGN KEY (Author_ID) REFERENCES Author(Author_ID),  
CONSTRAINT fMaterial FOREIGN KEY (Material_ID) REFERENCES  
Material(Material_ID)  
);
```

❖ **Creating Borrow Table**

```
CREATE TABLE Borrow (  
  Borrow_ID INT PRIMARY KEY,  
  Material_ID INT,  
  Member_ID INT,  
  Staff_ID INT,  
  Borrow_Date DATE ,  
  Due_Date DATE,  
  Return_Date DATE,  
  CONSTRAINT fk_material FOREIGN KEY (Material_ID) REFERENCES  
  Material(Material_ID),  
  CONSTRAINT fk_Member FOREIGN KEY (Member_ID) REFERENCES  
  Member(Member_ID),  
  CONSTRAINT fk_Staff FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID)
```

);

3.3 Populate the database with sample data. Using either an inset statement or importing directly as the file is accepted. Eight independent csv files consist of all data required for this project. You may insert the data into your database using any means if the content remains the same. You can remove quotes (") or commas (,) if necessary, other characters are not allowed to change.

❖ **Loading data into the Author table from Author.csv**

```
LOAD DATA INFILE 'D:/Sem 2/CS 504/Final PROJECT/Author.csv'  
  INTO TABLE Author  
  FIELDS TERMINATED BY ','  
  ENCLOSED BY '"'  
  LINES TERMINATED BY '\n'  
  IGNORE 1 ROWS;
```

❖ **Loading data into the Catalog table from Catalog.csv**

```
LOAD DATA INFILE 'D:/Sem 2/CS 504/Final PROJECT/Catalog.csv'  
  INTO TABLE Catalog  
  FIELDS TERMINATED BY ','  
  ENCLOSED BY '"'  
  LINES TERMINATED BY '\n'  
  IGNORE 1 ROWS;
```

❖ **Loading data into the Genre table from Genre.csv**

```
LOAD DATA INFILE 'D:/Sem 2/CS 504/Final PROJECT/Genre.csv'  
  INTO TABLE Genre  
  FIELDS TERMINATED BY ','  
  ENCLOSED BY '"'  
  LINES TERMINATED BY '\n'  
  IGNORE 1 ROWS;
```


❖ **Loading data into the Member table from Member.csv**

```
LOAD DATA INFILE 'D:/Sem 2/CS 504/Final PROJECT/Member.csv'
INTO TABLE Member
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

❖ **Loading data into the Staff table from Staff.csv**

```
INSERT INTO staff (Staff_ID, Name, Contact_Info, Job_Title, Hire_Date)
VALUES
(1, 'Amy Green', 'amy.green@email.com', 'Librarian', '2017-06-01'),
(2, 'Brian Taylor', 'brian.taylor@email.com', 'Library Assistant', '2018-11-15'),
(3, 'Christine King', 'chris.king@email.com', 'Library Assistant', '2019-05-20'),
(4, 'Daniel Wright', 'dan.wright@email.com', 'Library Technician', '2020-02-01');
```

❖ **Loading data into the Material table from Material.csv**

```
INSERT INTO material (Material, Title, Publication_Date, Catalog_ID, Genre_ID)
VALUES
(1, 'The Catcher in the Rye', '1951-07-16', 1, 1),
(2, 'To Kill a Mockingbird', '1960-07-11', 2, 1),
(3, 'The Da Vinci Code', '2003-04-01', 3, 2),
(4, 'The Hobbit', '1937-09-21', 4, 3),
(5, 'The Shining', '1977-01-28', 5, 4),
(6, 'Pride and Prejudice', '1813-01-28', 1, 1),
(7, 'The Great Gatsby', '1925-04-10', 2, 1),
(8, 'Moby Dick', '1851-10-18', 3, 1),
(9, 'Crime and Punishment', '1866-01-01', 4, 1),
(10, 'The Hitchhiker's Guide to the Galaxy', '1979-10-12', 5, 3),
(11, '1984', '1949-06-08', 1, 5),
(12, 'Animal Farm', '1945-08-17', 2, 5),
(13, 'The Haunting of Hill House', '1959-10-17', 3, 4),
(14, 'Brave New World', '1932-08-01', 4, 5),
(15, 'The Chronicles of Narnia: The Lion the Witch and the Wardrobe', '1950-10-16', 5, 3),
(16, 'The Adventures of Huckleberry Finn', '1884-12-10', 6, 1),
(17, 'The Catch-22', '1961-10-11', 7, 1),
(18, 'The Picture of Dorian Gray', '1890-07-01', 8, 1),
(19, 'The Call of Cthulhu', '1928-02-01', 9, 4),
(20, 'Harry Potter and the Philosopher's Stone', '1997-06-26', 10, 3),
(21, 'Frankenstein', '1818-01-01', 6, 4),
(22, 'A Tale of Two Cities', '1859-04-30', 7, 1),
(23, 'The Iliad', '1750-01-01', 8, 6),
(24, 'The Odyssey', '1725-01-01', 9, 6),
(25, 'The Brothers Karamazov', '1880-01-01', 10, 1),
(26, 'The Divine Comedy', '1320-01-01', 6, 6),
(27, 'The Grapes of Wrath', '1939-04-14', 7, 1),
(28, 'The Old Man and the Sea', '1952-09-01', 8, 1),
(29, 'The Count of Monte Cristo', '1844-01-01', 9, 1),
```

(30, 'A Midsummer Night's Dream', '1596-01-01', 10, 7),
(31, 'The Tricky Book', '1888-01-01', 10, 7);

❖ **Loading data into the Authorship table from Authorship.csv**

INSERT INTO authorship (Authorship_ID, Author_ID, Material_ID)

VALUES

(1, 1, 1),
(2, 2, 2),
(3, 3, 3),
(4, 4, 4),
(5, 5, 5),
(6, 6, 6),
(7, 7, 7),
(8, 8, 8),
(9, 9, 9),
(10, 10, 10),
(11, 11, 11),
(12, 12, 12),
(13, 13, 13),
(14, 14, 14),
(15, 15, 15),
(16, 16, 16),
(17, 17, 17),
(18, 18, 18),
(19, 19, 19),
(20, 20, 20),
(21, 1, 21),
(22, 2, 22),
(23, 3, 22),
(24, 3, 23),
(25, 4, 24),
(26, 5, 25),
(27, 6, 26),
(28, 7, 27),
(29, 8, 28),
(30, 19, 28),
(31, 9, 29),
(32, 10, 30),
(33, 8, 30),
(34, 2, 29);

❖ **Loading data into the Borrow table from Borrow.csv.**

```
INSERT INTO Borrow (Borrow_ID, Material_ID, Member_ID, Staff_ID, Borrow_Date, Due_Date, Return_Date)
```

```
VALUES
```

```
(1, 1, 1, 1, '2018-09-12', '2018-10-03', '2018-09-30'),  
(2, 2, 2, 1, '2018-10-15', '2018-11-05', '2018-10-29'),  
(3, 3, 3, 1, '2018-12-20', '2019-01-10', '2019-01-08'),  
(4, 4, 4, 1, '2019-03-11', '2019-04-01', '2019-03-27'),  
(5, 5, 5, 1, '2019-04-20', '2019-05-11', '2019-05-05'),  
(6, 6, 6, 1, '2019-07-05', '2019-07-26', '2019-07-21'),  
(7, 7, 7, 1, '2019-09-10', '2019-10-01', '2019-09-25'),  
(8, 8, 8, 1, '2019-11-08', '2019-11-29', '2019-11-20'),  
(9, 9, 9, 1, '2020-01-15', '2020-02-05', '2020-02-03'),  
(10, 10, 10, 1, '2020-03-12', '2020-04-02', '2020-03-28'),  
(11, 1, 11, 2, '2020-05-14', '2020-06-04', '2020-05-28'),  
(12, 2, 12, 2, '2020-07-21', '2020-08-11', '2020-08-02'),  
(13, 3, 13, 2, '2020-09-25', '2020-10-16', '2020-10-15'),  
(14, 4, 1, 2, '2020-11-08', '2020-11-29', '2020-11-24'),  
(15, 5, 2, 2, '2021-01-03', '2021-01-24', '2021-01-19'),  
(16, 6, 3, 2, '2021-02-18', '2021-03-11', '2021-03-12'),  
(17, 17, 4, 2, '2021-04-27', '2021-05-18', '2021-05-20'),  
(18, 18, 5, 2, '2021-06-13', '2021-07-04', '2021-06-28'),  
(19, 19, 6, 2, '2021-08-15', '2021-09-05', '2021-09-03'),  
(20, 20, 7, 2, '2021-10-21', '2021-11-11', NULL),  
(21, 21, 1, 3, '2021-11-29', '2021-12-20', NULL),  
(22, 22, 2, 3, '2022-01-10', '2022-01-31', '2022-01-25'),  
(23, 23, 3, 3, '2022-02-07', '2022-02-28', '2022-02-23'),  
(24, 24, 4, 3, '2022-03-11', '2022-04-01', '2022-03-28'),
```

(25, 25, 5, 3, '2022-04-28', '2022-05-19', '2022-05-18'),
 (26, 26, 6, 3, '2022-06-22', '2022-07-13', '2022-07-08'),
 (27, 27, 7, 3, '2022-08-04', '2022-08-25', '2022-08-23'),
 (28, 28, 8, 3, '2022-09-13', '2022-10-04', '2022-09-28'),
 (29, 29, 9, 3, '2022-10-16', '2022-11-06', '2022-11-05'),
 (30, 30, 8, 3, '2022-11-21', '2022-12-12', '2022-12-05'),
 (31, 1, 9, 4, '2022-12-28', '2023-01-18', NULL),
 (32, 2, 1, 4, '2023-01-23', '2023-02-13', NULL),
 (33, 3, 10, 4, '2023-02-02', '2023-02-23', '2023-02-17'),
 (34, 4, 11, 4, '2023-03-01', '2023-03-22', NULL),
 (35, 5, 12, 4, '2023-03-10', '2023-03-31', NULL),
 (36, 6, 13, 4, '2023-03-15', '2023-04-05', NULL),
 (37, 7, 17, 4, '2023-03-25', '2023-04-15', NULL),
 (38, 8, 8, 4, '2023-03-30', '2023-04-20', NULL),
 (39, 9, 9, 4, '2023-03-26', '2023-04-16', NULL),
 (40, 10, 20, 4, '2023-03-28', '2023-04-18', NULL);

4 Querying and Manipulation

4.1.1 Develop a set of SQL queries or stored procedures to perform common tasks, such as searching, updating, inserting, and deleting records in 4.1. You can only use a single SQL update or query statement for each question.

❖ **Search Example:** Search for Genre which are Fiction.

Query:

```
select * from genre Where Name Like "%Fiction";
```

Result:

	Genre_ID int	Name varchar(255)	Description text
> 1	1	General Fiction	"Literary works with a focus on character and plot development, exploring various themes and human experiences."
> 2	7	Historical Fiction	"Fictional stories set in the past, often based on real historical events or figures, and exploring the customs and experiences of t

❖ **Inserting Example:** Adding a new Genre

Query:

```
insert into genre (Name, Description) values ('Horror ', 'Genre of horror stories with comedic elements');
```

Result:

```
insert into genre (Name, Description) values ('Horror ', 'Genre of horror stories with comedic elements')  
  
AffectedRows : 1
```

❖ **Update Example:** Updating Name of Genre where Name is Horror

Query:

UPDATE Genre set Name='Horror Comedy' where Name='Horror ';

Result:

```
UPDATE Genre set Name='Horror Comedy' where Name='Horror '  
  
AffectedRows : 1
```

❖ **Delete Example:** Deleting the row from Genre where Name is Horror Comedy

Query:

delete FROM genre WHERE Name = 'Horror Comedy';

Result:

```
delete FROM genre WHERE Name = 'Horror Comedy'  
  
AffectedRows : 1
```

4.1.2 Demonstrate the use of advanced querying techniques, such as joins, aggregation, and subqueries.

Question 1: Calculate the total number of materials in each genre.

Aggregation Query:

```
SELECT g.Name, COUNT(m.Material_ID) AS Total_Materials  
FROM Genre g  
JOIN Material m ON g.Genre_ID = m.Genre_ID  
GROUP BY g.Name;
```

Result:

Q	Name varchar	Total_Materials bigint
> 1	General Fiction	14
> 2	Mystery & Thriller	1
> 3	Science Fiction & Fantasy	4
> 4	Horror & Suspense	4
> 5	Dystopian & Apocalyptic	3
> 6	Classics	3
> 7	Historical Fiction	2

Question 2: Retrieve the names of authors who have written materials in the "Science Fiction & Fantasy" genre.

Join Query:

SELECT a.Name

FROM Author a

JOIN Authorship au ON a.Author_ID = au.Author_ID

JOIN Material m ON au.Material_ID = m.Material_ID

JOIN Genre g ON m.Genre_ID = g.Genre_ID

WHERE g.Name = 'Horror & Suspense';

Result:

Q	Name varchar
> 1	J.K. Rowling
> 2	William Shakespeare
> 3	Fyodor Dostoevsky
> 4	Jane Austen

Question 3: Retrieve title of book and name of author which was published after 2000.

Sub Query:

```
SELECT m.Title, a.Name
```

```
FROM Material m
```

```
JOIN Authorship au ON m.Material_ID = au.Material_ID
```

```
JOIN Author a ON au.Author_ID = a.Author_ID
```

```
WHERE m.Title IN
```

```
( SELECT Title FROM Material WHERE Publication_Date > '2000-01-01' );
```

Result:

Q	Title varchar	Name varchar
> 1	The Da Vinci Code	George Orwell

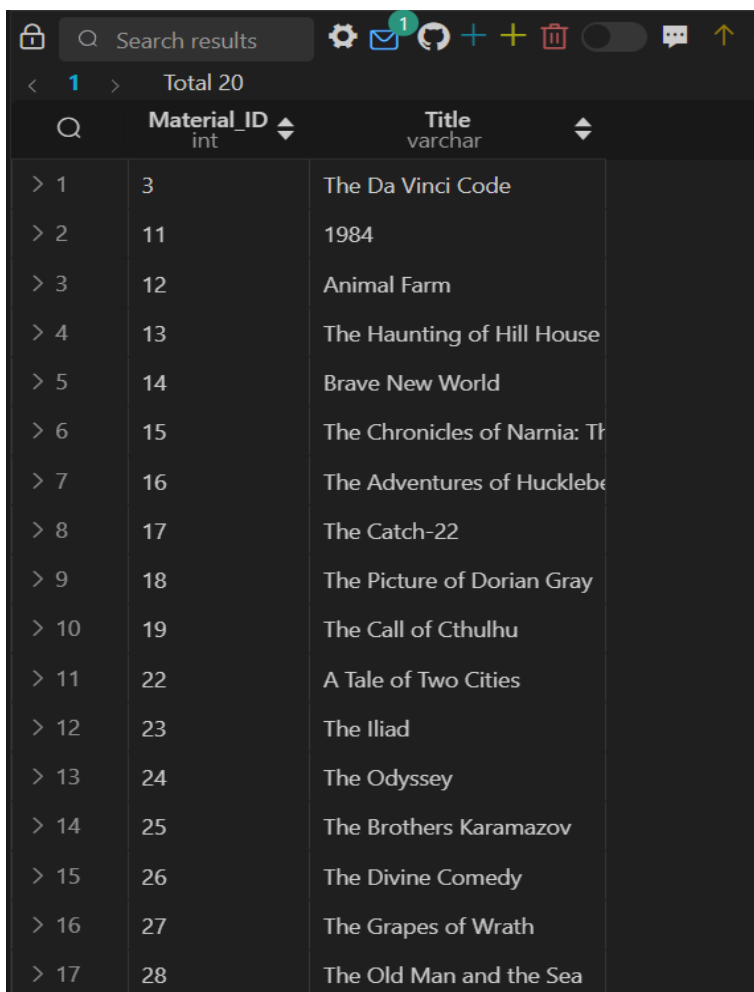
4.1. Queries/Updates

1. Which materials are currently available in the library? If a material is borrowed and not returned, it's not considered as available.

Query:

```
SELECT DISTINCT m.Material_ID, m.Title
FROM Material m
WHERE NOT EXISTS (
  SELECT 1
  FROM Borrow b
  WHERE b.Material_ID = m.Material_ID
  AND b.Return_Date IS NULL
);
```

Result:



The screenshot shows a database query result interface. At the top, there is a search bar with the text "Search results". Below it, a status bar indicates "Total 20" results. The main table has two columns: "Material_ID" (int) and "Title" (varchar). The table lists 17 materials, each with a unique ID and a title. The materials are: 3 (The Da Vinci Code), 11 (1984), 12 (Animal Farm), 13 (The Haunting of Hill House), 14 (Brave New World), 15 (The Chronicles of Narnia: The Lion, the Witch and the Wardrobe), 16 (The Adventures of Huckleberry Finn), 17 (The Catch-22), 18 (The Picture of Dorian Gray), 19 (The Call of Cthulhu), 22 (A Tale of Two Cities), 23 (The Iliad), 24 (The Odyssey), 25 (The Brothers Karamazov), 26 (The Divine Comedy), 27 (The Grapes of Wrath), and 28 (The Old Man and the Sea). Each row has a expandable icon (a right-pointing arrow) to its left.

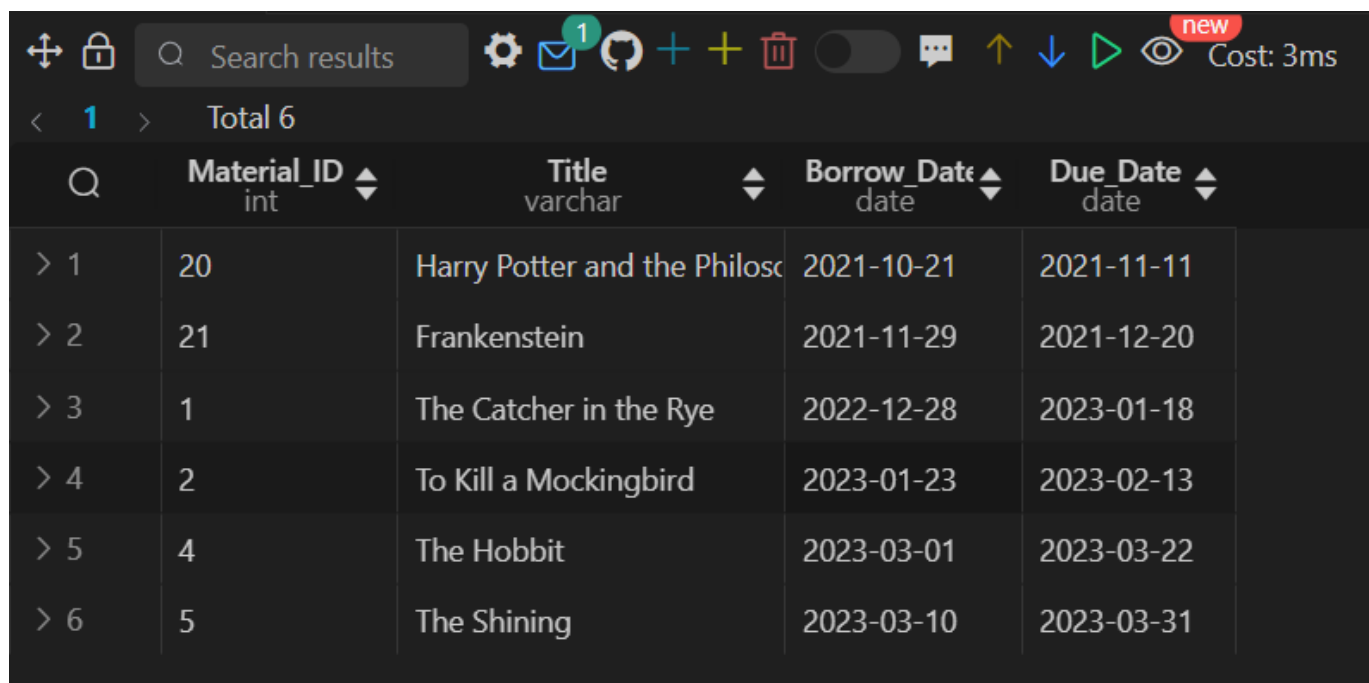
	Material_ID int	Title varchar
> 1	3	The Da Vinci Code
> 2	11	1984
> 3	12	Animal Farm
> 4	13	The Haunting of Hill House
> 5	14	Brave New World
> 6	15	The Chronicles of Narnia: The Lion, the Witch and the Wardrobe
> 7	16	The Adventures of Huckleberry Finn
> 8	17	The Catch-22
> 9	18	The Picture of Dorian Gray
> 10	19	The Call of Cthulhu
> 11	22	A Tale of Two Cities
> 12	23	The Iliad
> 13	24	The Odyssey
> 14	25	The Brothers Karamazov
> 15	26	The Divine Comedy
> 16	27	The Grapes of Wrath
> 17	28	The Old Man and the Sea

2. Which materials are currently overdue? Suppose today is 04/01/2023, and show the borrow date and due date of each material.

Query:

```
SELECT m.Material_ID, m.Title, b.Borrow_Date, b.Due_Date
FROM Material m
LEFT JOIN Borrow b ON m.Material_ID = b.Material_ID
WHERE b.Return_Date IS NULL
      AND b.Due_Date < '2023-04-01'
      AND b.Material_ID IS NOT NULL;
```

Result:



The screenshot shows a database query result interface. At the top, there is a search bar with the text "Search results" and a "Cost: 3ms" indicator. Below the search bar, there is a table with 5 columns: "Material_ID" (int), "Title" (varchar), "Borrow_Date" (date), and "Due_Date" (date). The table contains 6 rows of data, all of which are overdue as their due dates are before 2023-04-01. The interface includes various icons for search, settings, and navigation.

	Material_ID int	Title varchar	Borrow_Date date	Due_Date date
> 1	20	Harry Potter and the Philosc	2021-10-21	2021-11-11
> 2	21	Frankenstein	2021-11-29	2021-12-20
> 3	1	The Catcher in the Rye	2022-12-28	2023-01-18
> 4	2	To Kill a Mockingbird	2023-01-23	2023-02-13
> 5	4	The Hobbit	2023-03-01	2023-03-22
> 6	5	The Shining	2023-03-10	2023-03-31

3. What are the top 10 most borrowed materials in the library? Show the title of each material and order them based on their available counts.

Query:

```
SELECT m.Title, COUNT(b.Material_ID) AS Borrow_Count
FROM Material m
LEFT JOIN Borrow b ON m.Material_ID = b.Material_ID
GROUP BY m.Title
ORDER BY Borrow_Count DESC
LIMIT 10;
```

Result:

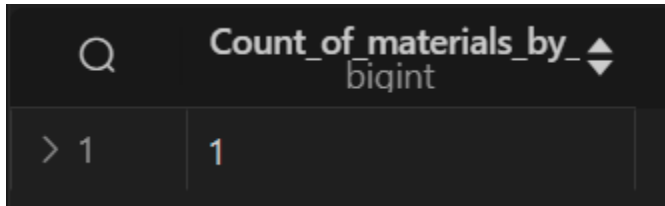
Q	Title varchar	Borrow_Count bigint
> 1	The Catcher in the Rye	3
> 2	To Kill a Mockingbird	3
> 3	The Da Vinci Code	3
> 4	The Hobbit	3
> 5	The Shining	3
> 6	Pride and Prejudice	3
> 7	The Great Gatsby	2
> 8	Moby Dick	2
> 9	Crime and Punishment	2
> 10	The Hitchhiker's Guide to th	2

4 How many materials has the author Lucas Piki written?

Query:

```
SELECT COUNT(*) AS Count_of_materials_by_Lucas_Piki
FROM Author a
JOIN Authorship au ON a.Author_ID = au.Author_ID
WHERE a.Name = 'Lucas Piki';
```

Result:



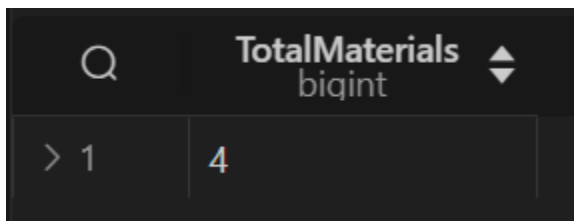
	Count_of_materials_by_ biqint
> 1	1

5 How many materials were written by two or more authors?

Query:

```
SELECT COUNT(DISTINCT Material_ID) AS TotalMaterials
FROM (
    SELECT a1.Material_ID
    FROM Authorship a1
    JOIN Authorship a2 ON a1.Material_ID = a2.Material_ID AND a1.Author_ID != a2.Author_ID
) AS subquery;
```

Result:



	TotalMaterials biqint
> 1	4

6 What are the most popular genres in the library ranked by the total number of borrowed times of each genre?

Query:

```
SELECT g.Name, COUNT(b.Material_ID) AS Borrow_Count
FROM Genre g
JOIN Material m ON g.Genre_ID = m.Genre_ID
```

JOIN Borrow b ON m.Material_ID = b.Material_ID

GROUP BY g.Name

ORDER BY Borrow_Count DESC;

Result:

Q	Name varchar	Borrow_Count bigint
> 1	General Fiction	22
> 2	Science Fiction & Fantasy	6
> 3	Horror & Suspense	5
> 4	Mystery & Thriller	3
> 5	Classics	3
> 6	Historical Fiction	1

7 How many materials had been borrowed from 09/2020-10/2020?

Query:

SELECT COUNT(*) AS Total_Borrowed_Materials

FROM Borrow

WHERE DATE(Borrow.Borrow_Date) BETWEEN DATE('2020-09-01') AND DATE('2020-10-31');

Result:

Q	Total_Borrowed_Materials bigint
> 1	1

8 How do you update the “Harry Poper and the Philosopher's Stone” when it is returned on 04/01/2023?

Query:

```
UPDATE Borrow
```

```
JOIN Material ON Borrow.Material_ID = Material.Material_ID
```

```
SET Borrow.Return_Date = '2023-04-01'
```

```
WHERE Material.Title = 'Harry Potter and the Philosopher"s Stone';
```

--check if the borrow is updated

```
SELECT b.*
```

```
FROM Borrow b
```

```
JOIN Material m ON b.Material_ID = m.Material_ID
```

```
WHERE m.Title = "Harry Potter and the Philosopher's Stone";
```

Result:

Q	Borrow_ID int	Material_ID int	Member_ID int	Staff_ID int	Borrow_Date date	Due_Date date	Return_Date date
> 1	20	20	7	2	2021-10-21	2021-11-11	2023-04-01

9 How do you delete the member Emily Miller and all her related records from the database?

Query1:

```
DELETE FROM Borrow
```

```
WHERE Member_ID IN (
```

```
    SELECT Member_ID
```

```
    FROM Member
```

```
    WHERE Name = 'Emily Miller'
```

```
);
```

```
SELECT * FROM Borrow
```

```
WHERE Member_ID IN (
```

```
    SELECT Member_ID
```

```
    FROM Member
```

```
WHERE Name = 'Emily Miller'
```

```
);
```

Query2:

```
DELETE FROM Member
```

```
WHERE Name = 'Emily Miller';
```

```
SELECT * FROM Member
```

```
WHERE Name = 'Emily Miller';
```

Result1:

	Borrow_ID int	Material_ID int	Member_ID int	Staff_ID int	Borrow_Date date	Due_Date date	Return_Date date

Result2:

		Member_ID int	Name varchar	Contact_Info varchar	Join_Date date

10. How do you add the following material to the database?

Title: New book

Date: 2020-08-01

Catalog: E-Books

Genre: Mystery & Thriller

Author: Lucas Luke

Query:

```
INSERT INTO Catalog (Name, Location)
```

```
VALUES ('E-Books', 'Online');
```

```
INSERT INTO Genre (Name, Description)
```

```
VALUES ('Mystery & Thriller', 'Genre of suspenseful and thrilling stories');
```

```
INSERT INTO Author (Name)
```

```
VALUES ('Lucas Luke');
```

```
SET @Material_ID = (SELECT MAX(Material_ID) + 1 FROM Material);
```

```
INSERT INTO Material (Material_ID, Title, Publication_Date)
```

```
VALUES (@Material_ID, 'New book', '2020-08-01');
```

```
SET @Authorship_ID = (SELECT MAX(Authorship_ID) + 1 FROM Authorship);
```

```
INSERT INTO Authorship (Authorship_ID, Material_ID)
```

```
VALUES (@Authorship_ID, @Material_ID);
```

Result:

```
INSERT INTO Catalog (Name, Location) VALUES ('E-Books', 'Online')
```

```
AffectedRows : 1
```

```
INSERT INTO Genre (Name, Description) VALUES ('Mystery & Thriller', 'Genre of suspenseful and thrilling stories')
```

```
AffectedRows : 1
```

```
INSERT INTO Author (Name) VALUES ('Lucas Luke')
```

```
AffectedRows : 1
```

```
INSERT INTO Material (Material_ID, Title, Publication_Date)
VALUES (@Material_ID, 'New book', '2020-08-01')
```

AffectedRows : 1

```
INSERT INTO Authorship (Authorship_ID, Material_ID)
VALUES (@Authorship_ID, @Material_ID)
```

AffectedRows : 1

5. Design of Extended Features

1) Alert staff about overdue materials on a daily basis?

Firstly, we need to figure out the materials that are overdue by comparing the current date with the due date of borrowed materials.

Sample SQL query:

```
SELECT bw.Borrow_ID, bw.Material_ID, bw.Member_ID, bw.Due_Date
FROM Borrow bw
WHERE bw.Return_Date IS NULL AND bw.Due_Date < CURDATE();
```

After identifying the overdue materials, Staff members who are responsible for overdue materials need to be extracted so that they can be informed regarding the issue.

Sample SQL query:

```
SELECT s.Name, s.Contact_Info, bw.Borrow_ID, bw.Material_ID, bw.Member_ID, bw.Due_Date
FROM Borrow bw
JOIN Staff s ON bw.Staff_ID = s.Staff_ID
WHERE bw.Return_Date IS NULL AND bw.Due_Date < CURDATE();
```

From the above two queries we can identify the overdue materials and the responsible staff members. To automate this process every day and to alert the staff about the overdue materials, A query can be deployed which executes every day and retrieves information regarding overdue materials. This can be possible by using Create, Event, Do, Begin, End and Select statements.

2) Automatically deactivate the membership based on the member's overdue occurrence ($\geq$ three times). And reactivate the membership once the member pays the overdue fee.

1. Firstly, new columns need to be created for showcasing the overdue count and the membership status. After creating the new columns, the columns need to be updated by the overdue count whether the material is returned after the due date.
2. If the overdue count is more than or equal to three then update the status column to "Deactive".
3. When a member pays the overdue fee, the overdue count needs to be updated to 0 and membership status to "Active".
4. To perform the above steps, Alter, Update, Trigger statements can be used in the SQL query.